

Visual Token Pruning and Text-Reading Performance in Vision-Language Models

Phase 3 Investigation using Qwen2.5-VL-7B

P. S. Kedar

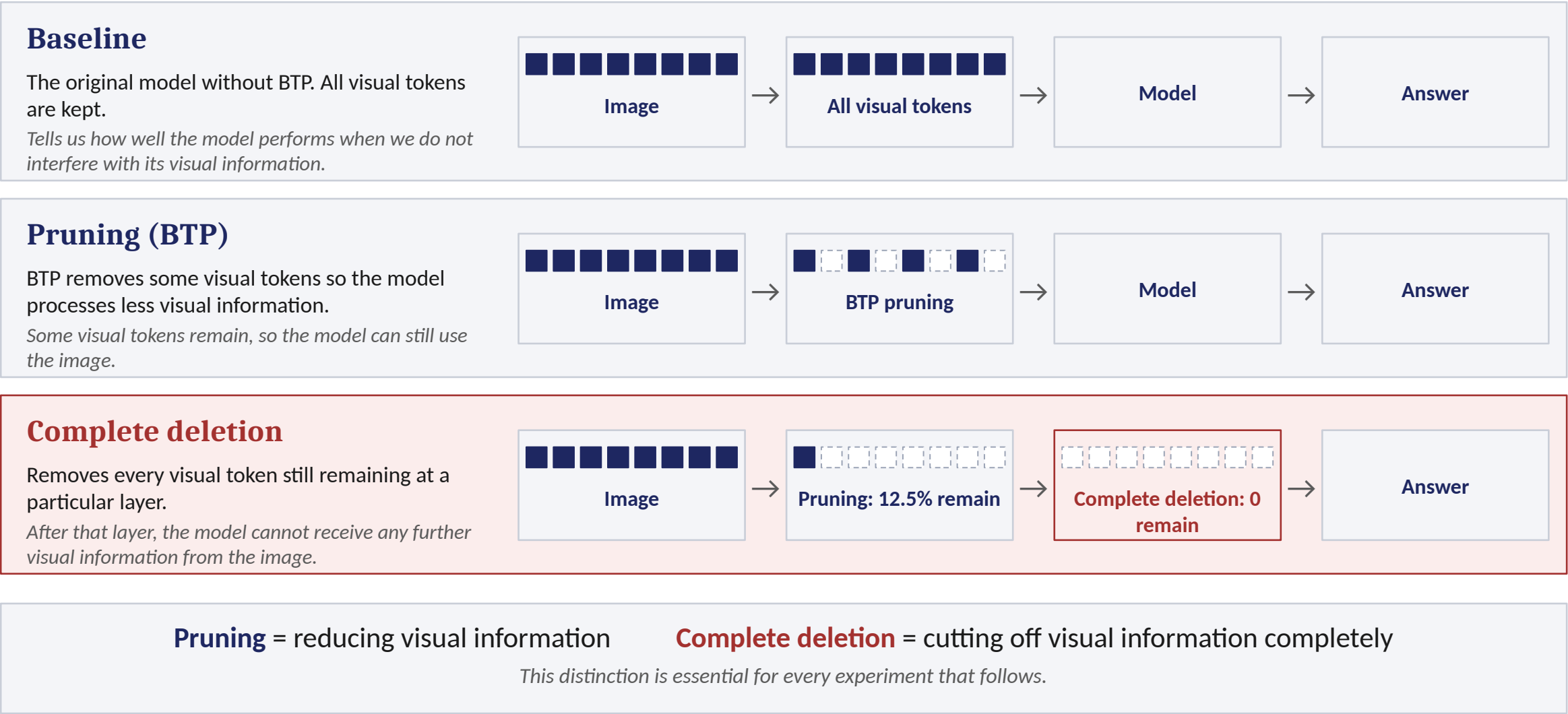
Supervised by Prof. Divya Saxena

Mentored by Aditya Sharma

IIT Jodhpur

Experimental Setup

An image is converted into many visual tokens before the language model processes it.



Initial Observation

We compared the original model (baseline) with BTP on Qwen2.5-VL-7B.

Task	Baseline	BTP
POPE	87.6	86.2
MMBench	83.7	79.3
GQA	60.9	55.9
AI2D	86.4	79.6
TextVQA *	86.2	23.3
DocVQA *	94.7	19.2
ChartQA *	76.8	29.0

* reading task

General visual tasks

POPE, MMBench, GQA, AI2D: performance decreases only moderately.

Reading tasks: performance collapses

Tasks where the model must identify and understand text physically present inside the image.

TextVQA questions answered from text in images

DocVQA understanding text in documents

ChartQA reading information from charts

Main observation

The degradation is not uniform. It is especially severe when the model must read text from the image.

Possible Causes

Why does BTP make Qwen fail on reading tasks? We tested four explanations.

1. Are we keeping the wrong visual tokens?

Why: Perhaps BTP removes useful tokens and keeps unhelpful ones.

What: Replaced BTP's token selection with other methods, including random selection.

Result: All methods stayed around 17–19% on TextVQA.

So what: Changing which tokens are selected does not recover performance.

2. Are text regions too similar or redundant?

Why: If text patches look alike, BTP may treat them as duplicates and remove them.

What: Compared how similar text patches are to each other versus background patches.

Result: Text similarity 0.38, background similarity 0.44.

So what: Text was not more redundant than background. This explanation was not supported.

3. Are too many tokens being removed?

Why: Removing a large share of tokens might destroy the information needed for reading.

What: Gradually changed token retention from 100% down to 12.5%.

Result: Even at 90% retention, TextVQA had already fallen to the same low level.

So what: The collapse happens even when almost all tokens are kept, so it is not the amount removed.

4. OCR-based text-token protection

Why: If BTP accidentally removes text, protecting the text tokens should restore performance.

What: OCR gives the text locations in the image. We forced the visual tokens at those locations to survive pruning, a best-case test.

Result: 19.7% with OCR-based protection, 19.7% without.

So what: Even explicitly protecting the text tokens does not recover performance.

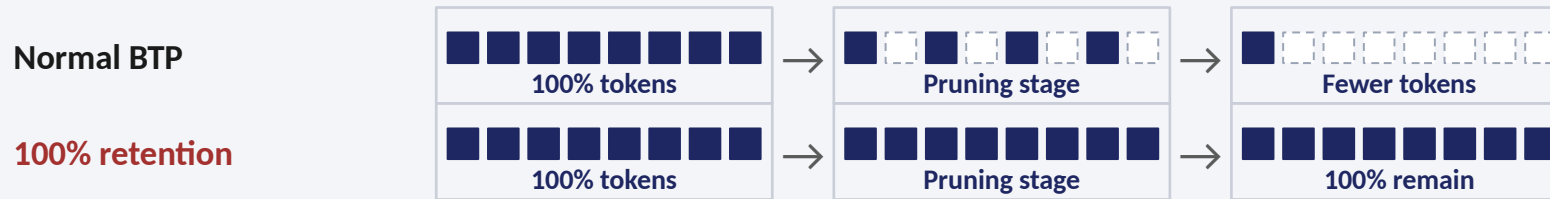
None of the four explanations can account for the collapse.

Control Experiment: 100% Token Retention

Is the pruning itself responsible for the collapse?

What does “BTP + 100% retention” mean?

We keep the BTP procedure active, but every pruning stage is set to retain 100% of the visual tokens. BTP runs, but nothing is removed by pruning.



Why: if pruning causes the collapse, removing the pruning should restore the original performance.

Condition	TextVQA
Baseline	87.6%
BTP + 100% retention	17.8%

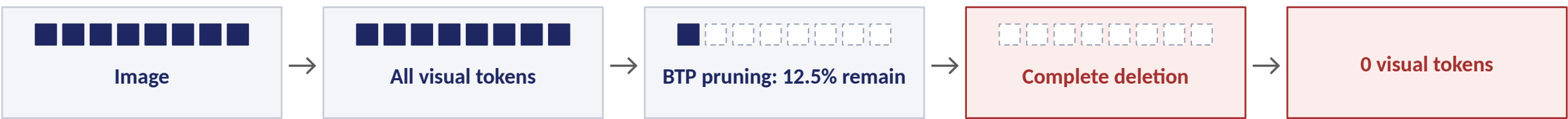
So what

The model still collapses even when no visual tokens are removed by pruning.

The major loss must be happening after the pruning stages.

Implementation Finding

What happens to the visual tokens after the pruning stages?



What we changed: kept the 12.5% pruning exactly the same, and switched off only the complete-deletion step.

Condition	TextVQA
No pruning	85.65%
BTP as released	17.25%
Complete deletion switched off	78.60%

Intuition

If pruning were responsible, switching off deletion would leave the score low. Instead it rises from 17.25% to 78.60%, with the pruning still in place.

Conclusion

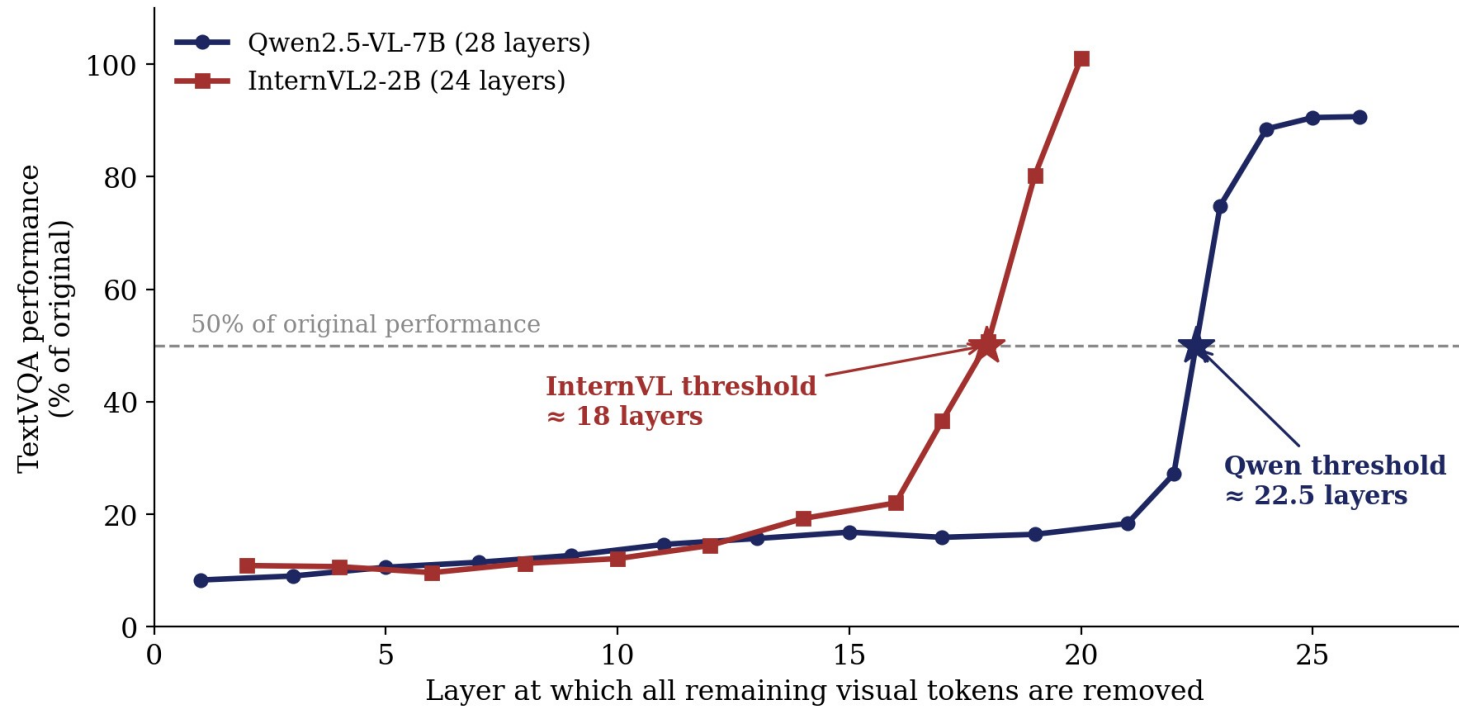
Most of the TextVQA collapse comes from completely removing the remaining tokens, not from the earlier pruning.

Note: the paper specifies retaining 12.5% of visual tokens for Qwen at the final stage, while the implementation tested here removes the remaining tokens.

Visual Token Access Threshold

Complete deletion at one layer causes a large drop. Does the exact layer matter?

What: we moved the complete-deletion point from early layers to later ones, and measured TextVQA at each point.



Threshold

The point where the model still keeps at least 50% of its original TextVQA performance.

Qwen2.5-VL-7B

≈ 22.5 of 28 layers → ≈ 80%

InternVL2-2B

≈ 18 of 24 layers → ≈ 75%

Relative threshold position: the threshold as a share of the model's layers.

Remove visual information too early → reading collapses. Keep it available longer → performance recovers sharply.

Measured on TextVQA, so these are task-specific measurements, not a universal law.

Pruning and Deletion Costs

Which part of the loss comes from pruning, and which from complete deletion?

Corrected BTP = normal BTP pruning to 12.5%, but without the complete-deletion step.

Pruning cost **Baseline → Corrected BTP**
This tells us the cost of removing some visual tokens.

Deletion cost **Corrected BTP → Released BTP**
This tells us the additional cost of removing all remaining tokens.

Dominant effect
This tells us whichever causes the larger share of the total drop.

Task	Pruning cost	Deletion cost	Dominant
TextVQA	5.7	57.2	Deletion
DocVQA	17.9	57.6	Deletion
ChartQA	31.4	16.4	Pruning
AI2D	7.2	0.4	Pruning
GQA	2.1	2.9	Both small
POPE	1.3	0.1	Both small
MMBench	4.7	0.3	Both small

Points of performance lost, Qwen2.5-VL-7B.

There is no single dominant effect for every task.
TextVQA and DocVQA: deletion dominates. ChartQA: pruning dominates. General tasks: both effects relatively small.

Evaluation Results

Why did the standard evaluation not clearly reveal this problem?

Configuration	Standard benchmarks	Reading tasks
Qwen, BTP as released	96.1%	28.4%
Qwen, deletion switched off	96.9%	77.9%
InternVL, pruning only	94.9%	56.5%

Share of each model's original performance retained.

Standard benchmarks
Around 95–97% retained, so pruning appears largely successful.

Reading tasks
Large losses become visible.

Important comparison: InternVL does not have the complete-deletion issue, yet its reading performance still drops substantially. The blind spot is not only caused by the Qwen implementation.

A benchmark suite without a suitable text-reading task can miss substantial degradation in visual reading ability.

Conclusion and Next Steps

Finding 1: Implementation

In the Qwen configuration tested, complete removal of the remaining visual tokens causes most of the TextVQA collapse.

Finding 2: Model behaviour

The model needs visual information to remain accessible up to a measurable point in the network. TextVQA threshold: Qwen $\approx$ 80%, InternVL $\approx$ 75%.

Finding 3: Evaluation

Standard benchmarks can show healthy overall performance while visual text-reading ability has degraded substantially.

Next steps

1. Measure the threshold on DocVQA and ChartQA
2. Repeat on a third architecture
3. Consolidate the results into the manuscript